Aplicaciones Móviles

Entender Apache Cordova

El problema que atiende, su contexto web y el recorrido de PhoneGap a Apache.

José Adrián Bazaldúa Huerta
Haili Ailin Ávila Ramírez
Universidad Iberoamericana

9 de noviembre de 2026

Un problema de plataformas

Una misma idea móvil podía exigir equipos, lenguajes y herramientas distintos para cada sistema operativo.

> El reto: llegar a más de una plataforma sin rehacer toda la interfaz.

01 Una experiencia para cada sistema

02 Código y herramientas diferentes

03 Más tiempo de mantenimiento

04 Necesidad de reutilizar

La web ya tenía una base común

HTML, CSS y JavaScript eran tecnologías conocidas para construir interfaces y comportamiento en el navegador.

> Una **WebView** lleva ese código web dentro de una aplicación para cada plataforma.

01 HTML estructura la pantalla

02 CSS define la presentación

03 JavaScript responde a acciones

04 La app se empaqueta por plataforma

La necesidad: web + dispositivo

Reutilizar código web no bastaba si la app no podía interactuar con capacidades del teléfono.

> Cordova une tecnologías web con **APIs del dispositivo**.

01 Sensores y estado de red

02 Cámara y almacenamiento

03 Interfaz web compartida

04 Acceso mediante APIs compatibles

2008: comienza PhoneGap

PhoneGap surgió como proyecto de código abierto de **Nitobi Software** en 2008.

> Su propuesta conectaba desarrollo web y aplicaciones móviles híbridas.

01 Nitobi inicia PhoneGap

02 Se publica como código abierto

03 Se enfoca en apps híbridas

04 Abre el camino del proyecto

2011: Adobe adquiere Nitobi

Adobe adquirió Nitobi en 2011 y, poco después, PhoneGap fue donado a la Apache Software Foundation.

> La donación separó la tecnología base de una sola empresa.

01 Adobe compra Nitobi

02 PhoneGap continúa como marca

03 La base se dona a Apache

04 Empieza una nueva etapa

Callback se convierte en Cordova

El proyecto entró a la incubadora de Apache en octubre de 2011 como **Callback**; el nombre pasó a ser Apache Cordova.

> Apache aporta un gobierno abierto basado en comunidad y contribuciones.

01 Octubre de 2011: incubación

02 Nombre inicial: Callback

03 Nombre del proyecto: Cordova

04 Proyecto Apache establecido en 2012

PhoneGap y Cordova no terminaron juntos

Adobe anunció el cierre de PhoneGap en 2020. Apache Cordova, como proyecto abierto mantenido por comunidad, continuó.

> **Cierre de PhoneGap** no significa cierre de Apache Cordova.

01 PhoneGap: producto y marca de Adobe

02 2020: Adobe anuncia su cierre

03 Cordova: proyecto de Apache

04 Su continuidad depende de su comunidad

Una historia que sigue en marcha

2008
Nitobi inicia PhoneGap.

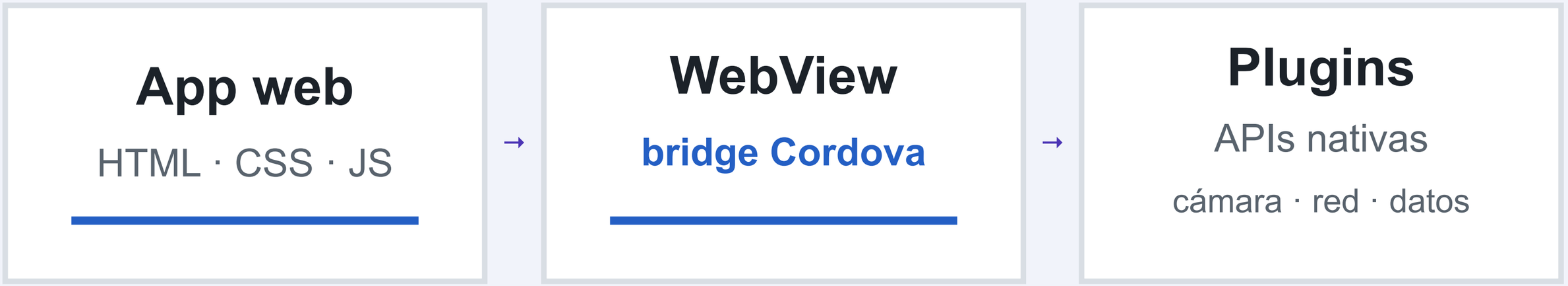
2011
Adobe adquiere Nitobi y la base llega a Apache.

2020
PhoneGap cierra; Cordova continúa.

2026
Android 15.1.0 e iOS 8.1.1 publicados.

> Las versiones publicadas documentan actividad; no miden por sí solas adopción ni cuota de mercado.

Arquitectura: capas que colaboran



La interfaz viaja como contenido web; los plugins conectan las llamadas JavaScript con código de cada plataforma.

La WebView ejecuta la interfaz

HTML, CSS y JavaScript corren dentro de una vista web integrada en el paquete de Android o iOS.

> No se convierte automáticamente en controles nativos: sigue siendo una interfaz web.

01 Renderiza pantallas y estilos

02 Ejecuta la lógica JavaScript

03 Carga recursos de `www/`

04 Comparte una base visual entre plataformas

Puente y plugins: el acceso al dispositivo

Un plugin expone una API JavaScript y la conecta con una implementación nativa para Android, iOS u otra plataforma.

> El puente permite pedir una capacidad; el plugin define cómo atenderla en cada sistema.

01 JavaScript invoca una API

02 El bridge entrega el mensaje

03 El plugin ejecuta código nativo

04 El resultado vuelve a JavaScript

Archivos que organizan un proyecto

`config.xml`
identidad, preferencias y configuración Cordova.

`www/`
HTML, CSS, JavaScript y recursos de la app.

`platforms/`
proyectos nativos generados; no es la fuente principal.

`plugins/`
integraciones instaladas para capacidades del dispositivo.

> También `package.json` declara dependencias y plataformas del proyecto.

El CLI coordina el flujo, no reemplaza la revisión



> Es un mapa conceptual: cada plataforma conserva requisitos, permisos y firmas propios.

QueVer: dos recorridos distintos

Buscar: JavaScript consulta OMDb por HTTPS y dibuja los resultados.

> **Guardar:** JavaScript llama al plugin SQLite y Android conserva los datos.

01 HTML y CSS: pantalla

02 JavaScript: acciones y API

03 Cordova: contenedor y puente

04 SQLite: catálogo local

Usos y límites: una decisión situada

Puede encajar en flujos internos, catálogos o productos que parten de una base web y requieren capacidades concretas del dispositivo.

> La reutilización no elimina la evaluación de rendimiento, seguridad, permisos y mantenimiento de plugins.

01 Equipos con base web existente

02 Necesidades nativas bien delimitadas

03 Soporte real de los plugins elegidos

04 Cambios de Android e iOS a vigilar

Alternativas: comparar el modelo, no buscar un ganador

Opción	Interfaz principal	Acceso a plataforma	Pregunta útil
Cordova	WebView: HTML, CSS y JS	Plugins y bridge	¿La app web y los plugins cubren el alcance?
Capacitor	Web dentro de una app nativa	Plugins y APIs de Capacitor	¿Conviene su integración con el stack web actual?
React Native	Componentes React renderizados de forma nativa	Módulos nativos	¿El equipo trabaja con React y necesita UI nativa?
Flutter	Widgets renderizados por Flutter	Plugins y canales de plataforma	¿Dart y su modelo visual se ajustan al producto?

Conclusión: una capa web con responsabilidades nativas

Cordova permite empaquetar una aplicación web y conectar capacidades del dispositivo mediante una WebView, un puente y plugins.

> Su valor está en la reutilización consciente; su límite está en todo lo que la plataforma y los plugins aún exigen mantener.

1. Apache Cordova. *Overview*, CLI Guide, Project Structure, Plugin Development y Security Guide. cordova.apache.org/docs/en/latest/
2. Apache Cordova. "Goodbye PhoneGap" (2020) y anuncios Android 15.1.0 / iOS 8.1.1 (2026). cordova.apache.org/announcements/
3. Capacitor. *Documentation*. capacitorjs.com/docs
4. React Native. *Introduction*. reactnative.dev/docs/getting-started
5. Flutter. *Architectural overview*. docs.flutter.dev/resources/architectural-overview

Continúa en la práctica Android: seis fases y archivos completos. Guion: 12 minutos de teoría.